

ffuf

(Fuzzing Faster U Fool)

Practical Web Fuzzing Notes

Structured directly from the ffuf commands practiced in web-security labs

Scope

These examples use private lab IP addresses. Use ffuf only against systems you own or targets where you have explicit permission to test.

From URL discovery -> extensions -> directories -> files -> parameters -> IDs -> POST data -> proxying

1. Introduction to ffuf

ffuf stands for “**Fuzz Faster U Fool.**” It is a fast web fuzzer written in Go and is commonly used for discovering web content and testing different parts of HTTP requests. Its basic idea is simple: give ffuf many candidate values from a wordlist, put each value where you place the FUZZ keyword, send the requests, and inspect which responses look interesting.

The core mental model

WORDLIST -> FUZZ -> HTTP request -> HTTP response -> MATCH / FILTER -> interesting result

What does “fuzzing” mean here?

In these labs, fuzzing means systematically replacing one part of an HTTP request with many candidate inputs. For example, when the URL is `http://target/FUZZ` and the wordlist contains `admin`, `login`, `config`, `docs`, ffuf can test each of those as a possible path.

The FUZZ keyword

FUZZ is the placeholder that tells ffuf where to insert a wordlist value. The keyword can appear in the URL, headers, or request body. In the commands you practiced, it appears mainly in paths, query parameters, and POST data.

Match vs filter

A matcher says “keep responses that look like this.” A filter says “remove responses that look like this.” For example, `-mc 200` keeps HTTP 200 responses, while `-fw 39` removes responses containing exactly 39 words. ffuf first applies matchers, then filters.

2. How to read ffuf output

Field	Meaning
Status	HTTP response status code, such as 200, 301, 302, 403.
Size	Response size/content length reported by ffuf.
Words	Number of whitespace-separated words in the response.
Lines	Number of lines in the response.
Duration	Time taken for that request.
Progress	How many wordlist entries ffuf has processed.

Important

A result is not automatically a vulnerability. It is a lead. You normally open the discovered path manually, inspect the response, and decide whether it is real, useful, or a false positive.

Query 1: Basic directory/content discovery

Goal: Find web paths that return HTTP 200.

```
ffuf -u http://10.48.179.191/FUZZ -w /usr/share/seclists/Discovery/Web-Content/big.txt:FUZZ -mc 200
```

Break the command down

Part	Meaning
ffuf	Run the ffuf web fuzzer.
-u <URL>	Target URL. ffuf sends requests to this URL.
FUZZ	Placeholder where each wordlist entry will be inserted.
-w <wordlist>:FUZZ	Load a wordlist and explicitly name its keyword FUZZ. FUZZ is also ffuf's default keyword, so this explicit name is valid but not strictly necessary.
-mc 200	Match only HTTP 200 responses.

What happened?

If the wordlist contains README.md, ffuf effectively requests `http://10.48.179.191/README.md`. It repeats this for every entry. Because `-mc 200` is enabled, only responses with status 200 appear in the result list.

How to interpret your results

Finding	Interpretation
README.md — 200	A resource at /README.md responded successfully.
favicon.ico — 200	A favicon exists; usually useful as confirmation that a normal web resource is present, but rarely a major finding.
robots.txt — 200	The file exists and may reveal paths the site operator does not want crawlers to index.

New idea learned

A very basic ffuf scan can be used as content discovery: wordlist entry -> URL path -> HTTP request -> status-based result.

Query 2: Directory discovery + extension testing

Goal: Look for directories while also testing .txt variants.

```
ffuf -u http://10.48.179.191/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-medium-words-lowercase.txt -e .txt
```

Already known

You already know `-u`, `-w`, and `FUZZ`. The new pieces here are `-e` and the fact that no `-mc` matcher was supplied.

New concept 1: `-e`

`-e` adds file extensions to the `FUZZ` keyword. For example, with a word such as `config`, an extension-based run can test variants such as `config` and `config.txt`. The `ffuf` documentation defines `-e` as a comma-separated list of extensions that extend the `FUZZ` keyword.

New concept 2: no `-mc`

You did not specify `-mc` here, so `ffuf` uses its normal default matching set rather than restricting the scan to only 200. That is why results such as 301 and 403 can appear.

Your useful findings

Result	What it tells you
config — 301	The path exists and the server redirects it, commonly toward <code>/config/</code> . A 301 is often useful during directory discovery.
docs — 301	A docs directory/path likely exists.
external — 301	An external-related directory/path likely exists.
robots.txt — 200	The file exists and returned successfully.
403 entries	The server responded with Forbidden. This can still be a useful lead, although it must be validated because servers can return generic 403 responses.

Think about status codes

200 = success. 301/302 = redirect. 403 = server understood the request but refuses access. During enumeration, redirects and 403 responses can still reveal that a path is interesting.

Query 3: Fuzzing a filename suffix with an extension wordlist

Goal: Test which extension belongs to index.

```
ffuf -u http://10.48.179.191/indexFUZZ -w /usr/share/seclists/Discovery/Web-Content/web-extensions.txt
```

Already known

You already know `-u`, `-w`, `FUZZ`, and response matching. The new idea is where `FUZZ` is placed.

New concept: `FUZZ` does not have to replace the whole path

`FUZZ` can be placed inside a larger string. Here the target is `indexFUZZ`. If the wordlist contains `.php`, `ffuf` effectively requests `/index.php`. If it contains `.phps`, it requests `/index.phps`.

Your result

Result	Interpretation
index.php — 302	The resource exists and redirects. This strongly suggests index.php is an active application entry point.
index.phps — 403	The server returned Forbidden for this variant. It is not the same as confirming that the file is readable.

Key takeaway

Fuzzing is about choosing the correct location for FUZZ. You can fuzz a whole path, a suffix, a parameter name, a parameter value, a header, or request-body data.

Query 4: Directory wordlist on a different lab target

Goal: Enumerate likely directory names using a directory-focused wordlist.

```
ffuf -u http://10.49.173.75/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-medium-directories-lowercase.txt
```

What is new?

The command structure is almost the same as Query 2. The main new learning is wordlist selection: different wordlists are built for different discovery tasks.

Wordlist idea

A directory-focused wordlist is intended to contain likely directory names. A file-focused list contains likely filenames. An extensions list contains suffixes such as .php. The choice of wordlist changes what questions you are asking the target.

Results

Finding	Meaning
docs — 301	Likely directory; redirect commonly indicates the slash-terminated directory URL.
config — 301	Likely directory/path named config.
external — 301	Likely directory/path named external.
server-status — 403	Potentially interesting administrative/status endpoint, but access is forbidden.

New skill

Do not think of “ffuf scan” as one fixed command. Think in terms of the discovery question: directories? files? extensions? parameter names? values? Each question gets a suitable wordlist and FUZZ placement.

Query 5: File discovery with filters

Goal: Find useful files while removing generic 403 responses and testing a response regex.

```
ffuf -u http://10.49.173.75/FUZZ -w /usr/share/seclists/Discovery/Web-Content/raft-medium-files-lowercase.txt -fr '/\..*' -fc 403
```

New concept 1: -fc

-fc means “filter status code.” -fc 403 tells ffuf to hide every response whose HTTP status is 403. Unlike -mc, which keeps matching responses, -fc removes matching responses from the displayed findings.

New concept 2: -fr

-fr means “filter by regular expression.” The regex is applied to the HTTP response headers and body. It is not a filter on the discovered URL or filename.

Very important correction about '/\..*'

The pattern '/\..*' means “a slash, followed by a literal dot, followed by any characters.” Because -fr operates on response headers/body, this pattern is NOT a general instruction to remove filenames containing dots. If the goal was simply to hide 403 results, -fc 403 is the relevant filter.

Why did wp-forum.phps still appear?

Your own output shows wp-forum.phps with status 403. That is exactly the kind of response -fc 403 is supposed to remove. If it appeared in a run where -fc 403 was not actually active, or if the command differed, the result can still show up. In general, verify the command summary at the top of ffuf output before interpreting a finding.

Rule to remember

-mc / -fc = status-based matching/filtering. -ms / -fs = response-size matching/filtering. -mw / -fw = word-count matching/filtering. -ml / -fl = line-count matching/filtering. -mr / -fr = regex matching/filtering.

Query 6: Discovering GET parameter names

Goal: Find which query-string parameter name changes the application response.

```
ffuf -u 'http://10.48.155.74/sqli-labs/Less-1/?FUZZ=1' -c -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt -fw 39
```

New concept 1: parameter fuzzing

This time FUZZ is not a path. It is the parameter NAME in the query string. The template is ?FUZZ=1. ffuf can therefore turn wordlist entries into requests such as ?id=1, ?user=1, ?page=1, and so on. The official ffuf documentation describes this as GET parameter-name fuzzing.

New concept 2: -fw

-fw filters responses by word count. Here you told ffuf to hide responses containing exactly 39 words. This is useful when invalid parameter names all return the same generic page, while a valid parameter causes the application to respond differently.

New concept 3: -c

-c enables colored terminal output. It does not change the requests sent to the target; it only makes the terminal result easier to read.

Why id appeared

Most parameter names produced the baseline response that you filtered out at 39 words. The id parameter produced a different response (37 words), so it survived the filter. That does not by itself prove SQL injection; it identifies a parameter worth testing.

Important mental shift

Here ffuf is not asking “what pages exist?” It is asking “which parameter name changes application behavior?”

Query 7: Enumerating valid numeric IDs

Goal: Use a generated numeric wordlist to discover which ID values are valid.

```
for i in {0..255}; do echo $i; done | ffuf -u 'http://10.48.155.74/sqli-labs/Less-1/?id=FUZZ' -c -w - -fw 33
```

New concept 1: generating the wordlist with Bash

The left side creates numbers from 0 through 255. `for i in {0..255}` loops over that numeric range, and `echo $i` prints the current number.

New concept 2: piping into ffuf

The `|` symbol sends the generated numbers to ffuf through standard input. This means you do not need to save the 256 values in a separate file.

New concept 3: -w -

A dash as the wordlist source tells ffuf to read wordlist entries from standard input. So ffuf receives 0, 1, 2, ... 255 from the pipe.

New concept 4: -fw 33

You filtered the generic 33-word response. Values that caused a different response survived the filter and were printed as likely valid IDs.

How to read the result

Your output shows IDs including 1, 2, 3, 4, 5, ... 14. Within the tested range, 14 was the highest valid ID that appeared in your results.

Why did the output order look random?

ffuf runs multiple requests concurrently (your output shows 40 threads). Because requests finish at different times, findings do not have to print in numeric order.

Do we know that 14 is the globally highest ID?

No. We only know that 14 was the highest matching value in the range you tested, 0-255. If the application had a valid record at 300, this command would never test it. The numeric range is your test scope, not a property of ffuf.

Connection to the previous query

Query 6 discovered the parameter name: id. Query 7 then fuzzed the value of that known parameter: id=FUZZ. This is a very common progression in web testing: first identify the input location, then test values in that location.

Practical pattern

Unknown parameter name -> FUZZ the name. Known parameter name -> FUZZ the value.

Query 8: POST parameter fuzzing for a lab login form

Goal: Test password candidates in a POST request and identify the response that differs from the normal failure page.

```
ffuf -u http://10.48.155.74/sqli-labs/Less-11/ -c -w /usr/share/seclists/Passwords/Leaked-Databases/hak5.txt -X POST -d 'uname=Dummy&passwd=FUZZ&submit=Submit' -fs 1435 -H 'Content-Type: application/x-www-form-urlencoded'
```

New concept 1: -X POST

-X selects the HTTP method. Here you are sending a POST request instead of the default GET request.

New concept 2: -d

-d supplies the POST request body. FUZZ is placed inside passwd, so every wordlist entry becomes a different password candidate.

New concept 3: -H

-H adds an HTTP header. The Content-Type tells the server that the body is URL-encoded form data, which matches the normal format used by a typical HTML login form.

New concept 4: -fs

-fs filters by response size. You observed that failed login attempts had a response size of 1435 bytes, so -fs 1435 removes the normal failure response and leaves responses with a different size. The ffuf documentation defines -fs as a response-size filter.

What the successful-looking result means

p@ssword produced a response of 1526 bytes instead of the 1435-byte baseline. In this lab, that difference identified the password candidate that changed the application response.

Why response size is useful

Many applications return the same page for every incorrect input. Instead of reading thousands of identical responses, you can establish a baseline and filter it out with `-fs`, `-fw`, or another suitable filter.

Lab-only note

Password wordlists and automated login testing should be used only against an application you are explicitly authorized to test, such as a dedicated training lab.

Query 9: Sending ffuf traffic through an HTTP proxy

Goal: Route ffuf requests through a local HTTP proxy so the traffic can be inspected or handled by another tool.

Terminal 1: start a local HTTP proxy

```
ncat -l 8080 --proxy-type http
```

Terminal 2: send ffuf traffic through the proxy

```
ffuf -u http://10.49.145.57/FUZZ -c -w /usr/share/seclists/Discovery/Web-Content/common.txt -x  
http://127.0.0.1:8080
```

New concept 1: ncat listen mode

`ncat -l 8080` tells Ncat to listen for incoming connections on local port 8080. The `-l` option is listen mode.

New concept 2: Ncat as an HTTP proxy

With `--proxy-type http` in listen mode, Ncat acts as an HTTP proxy server. Ncat's official guide documents the pattern `ncat -l <port> --proxy-type http` for creating a temporary HTTP proxy.

New concept 3: ffuf -x

`-x` sets the proxy URL for ffuf. Here `127.0.0.1:8080` means "send the ffuf requests to the local proxy listening on port 8080." ffuf supports proxy URLs through `-x`.

Traffic flow

Request path

ffuf -> 127.0.0.1:8080 (Ncat HTTP proxy) -> 10.49.145.57 -> web server -> response -> back through proxy -> ffuf

This technique is useful when you want ffuf traffic to pass through a proxy that can observe, modify, or log requests. In this exact command, the proxy is Ncat itself; if you want a graphical HTTP-testing workflow, a dedicated intercepting proxy can be used instead.

3. What you learned across all nine queries

Task	FUZZ location	Main technique
Directory/content discovery	/FUZZ	Wordlist + status matcher
Extensions	/FUZZ + -e	Test extension variants
Filename suffix	/indexFUZZ	Put FUZZ inside a fixed path
Directory enumeration	/FUZZ	Choose a directory-focused wordlist
File enumeration + filters	/FUZZ	Use -fc / -fr to reduce noise
GET parameter names	?FUZZ=1	Discover input names
Parameter values / IDs	?id=FUZZ	Generate or supply candidate values
POST data	passwd=FUZZ	Fuzz request-body input
Proxying	Any location	Send ffuf requests via -x

4. Matcher and Filter cheat sheet

Flag / Syntax	What it does	Why it matters
-mc 200	Match HTTP status 200.	Keep only successful 200 responses.
-fc 403	Filter HTTP status 403.	Hide forbidden responses.
-ms 1234	Match response size 1234 bytes.	Keep only responses of that size.
-fs 1435	Filter response size 1435 bytes.	Remove common baseline responses.
-mw 37	Match 37 words.	Keep responses by word count.
-fw 39	Filter 39 words.	Hide generic responses of that word count.
-mr REGEX	Match response headers/body with a regex.	Keep responses containing a pattern.
-fr REGEX	Filter response headers/body with a regex.	Remove responses containing a pattern.
-ml / -fl	Match/filter by line count.	Useful when line count is more stable than size.

Simple rule

Matchers keep. Filters remove.

5. A practical ffuf workflow

1. Identify the discovery question: path, file, extension, parameter name, parameter value, POST field, or header.
2. Choose a wordlist that fits that question.
3. Place FUZZ exactly where the changing input belongs.
4. Run a small test first and inspect the normal response.
5. Choose a matcher or filter based on how the application behaves.
6. Review interesting results manually. A status code or response-size difference is a lead, not proof of a vulnerability.

The most important habit

Before adding a filter, establish a baseline. Send one obviously invalid request, record its status/size/word count/lines, and then choose the filter that removes that normal response.

6. Common mistakes to avoid

- Mistaking a 403 for a confirmed vulnerability. It only tells you the server returned Forbidden.
- Using -fr as if it filters filenames or URLs. It filters response headers/body by regex.
- Assuming the highest ID found is the highest ID that exists. It is only the highest match within the range you tested.
- Choosing a random wordlist. The wordlist should match the discovery question.
- Forgetting that -mc and -fc have opposite purposes: one keeps, the other removes.
- Treating every ffuf hit as important. Many hits are noise or generic application responses.
- Running automated fuzzing against systems without authorization.

7. Notes from your actual lab outputs

Observation	Lesson
README.md and robots.txt returned 200	Basic content discovery can find ordinary web files.
config/docs/external returned 301	Redirects are useful discovery signals, especially for directories.
index.php returned 302	The FUZZ keyword can be used as a suffix to identify a file extension.
id survived the 39-word filter	Parameter-name fuzzing can reveal which input changes the application response.
IDs 1-14 appeared as matches	Numeric values can be generated on the fly and tested without a saved wordlist.
p@ssword changed response size	Response-size filtering can separate a different login result from the generic failure page.
ffuf output showed 40 threads	Concurrent requests mean output order may not match input order.
Ncat on 127.0.0.1:8080	ffuf can route traffic through a local HTTP proxy.

8. One-page memory summary

ffuf in one sentence

ffuf takes many candidate inputs, inserts them at FUZZ, sends HTTP requests, and uses matchers/filters to help you spot responses that differ from the normal baseline.

Flag / Syntax	What it does	Why it matters
-u	Target URL	Where the request goes
-w	Wordlist / input source	What ffuf tries
FUZZ	Replacement keyword	Where ffuf inserts each input
-e	Extensions	Try extension variants
-mc	Match status	Keep selected statuses
-fc	Filter status	Remove selected statuses
-fs	Filter size	Remove common-size responses
-fw	Filter words	Remove common-word-count responses
-fr	Filter regex	Remove responses matching a regex
-X	HTTP method	Switch to POST, etc.
-d	Request body	Fuzz form/body data
-H	HTTP header	Send required/custom headers
-x	Proxy	Route traffic through a proxy
-c	Color output	Make terminal output easier to read

9. References

- ffuf official GitHub repository and README: <https://github.com/ffuf/ffuf>
- ffuf official Wiki - matching and filtering: <https://github.com/ffuf/ffuf/wiki>
- Ncat official Users' Guide - proxying: <https://nmap.org/ncat/guide/ncat-proxy.html>
- Ncat official reference guide: <https://nmap.org/book/ncat-man.html>